Building a Local LLM Serving Lab

After building a backend API and async worker system, the next natural layer in an AI infrastructure learning path is model serving.

The async backend project answered one foundational question: how should a platform accept, track, and process long-running work reliably?

The Local LLM Serving Lab answers a different but equally important question: how should a platform expose language model inference as a service?

This distinction matters. Running a model locally is useful, but it is not the same as designing model-serving infrastructure. A local model running through a command-line tool proves that inference is possible. A model-serving layer proves that inference can be wrapped, called, streamed, measured, compared, and eventually connected to larger AI platform workflows.

That is the purpose of the `03-local-llm-serving-lab`.

It moves beyond “I can run a model locally” and starts treating local inference like infrastructure.

The project uses FastAPI, Ollama, a mock runtime, streaming endpoints, runtime comparison notes, and benchmark tooling to create a small but meaningful local model-serving environment. From an AI Data Platform Architect’s perspective, this is an important step because model serving is one of the core platform capabilities that sits underneath RAG systems, AI agents, evaluation pipelines, coding assistants, and internal AI tools.

The model is only one part of the system.

The platform around the model is what makes it usable.

Why This Project Exists

Local LLM tooling has become accessible enough that a developer can run capable models on a laptop using tools like Ollama, llama.cpp, and small instruction-tuned models.

That accessibility is valuable, but it can also create a false sense of completeness.

Running a model locally does not automatically teach model-serving architecture.

A production-oriented model-serving mindset requires deeper questions:

How does an application call the model?

What API contract should sit in front of the runtime?

How should prompts be submitted?

How should model names, temperature, and token limits be configured?

How should the service expose health?

How should it list available models?

How should it stream output?

How should latency be measured?

How should throughput be measured?

How should one runtime be compared against another?

How should the system support future RAG, observability, load testing, and deployment work?

The Local LLM Serving Lab exists to answer those questions in a controlled, local-first environment.

It is intentionally practical. Instead of jumping directly to Kubernetes, GPUs, distributed serving, or enterprise inference gateways, the lab starts with a clear local architecture: a FastAPI wrapper that sits between clients and local model runtimes.

That shape is simple, but it teaches a critical platform lesson: model inference should be exposed through a stable service interface.

The System at a Glance

The lab is centered on a FastAPI wrapper.

That wrapper sits between the client and the local model runtime.

The client sends a prompt to the API.

The API validates and routes the request.

The runtime client selects the backend.

The local model runtime performs inference.

The API returns either a full response or streamed chunks.

The benchmark script sends repeated requests and records performance.

The major components are:

- **Client or app:** sends prompts and receives full or streamed responses.
- **FastAPI wrapper:** exposes the model-serving API.
- **Runtime client:** abstracts the selected inference backend.
- **Local model runtime:** performs tokenization and generation.
- **Benchmark script:** measures latency, throughput, and output characteristics.

This architecture is valuable because it separates application concerns from runtime concerns.

A client should not need to know whether the response came from Ollama, llama.cpp, a future GPU-backed server, or a mock runtime. The client should call one API contract.

That is the architectural principle: hide runtime complexity behind a stable model-serving interface.

Why FastAPI Works Well for the Serving Wrapper

FastAPI is a strong fit for this kind of lab because it gives the model-serving layer a clean HTTP surface.

The API does not need to be large. In fact, one of the strengths of the project is that the endpoint surface is intentionally focused.

The lab exposes:

Endpoint	Purpose
`GET /health`	Check wrapper and runtime status
`GET /models`	List available local models
`POST /generate`	Return a complete generated response
`POST /generate/stream`	Stream generated text chunks as they arrive
`GET /docs`	Explore generated OpenAPI documentation

This is a compact but useful model-serving API.

`GET /health` gives the system an operational surface. It allows a developer or future orchestrator to check whether the wrapper and runtime are available.

`GET /models` exposes available local models. This matters because model selection is part of inference configuration.

`POST /generate` supports traditional request-response generation. The client sends a prompt and receives a complete response.

`POST /generate/stream` supports incremental generation. The client begins receiving output before the full answer is complete.

`GET /docs` gives the service self-documenting API behavior through OpenAPI.

From a platform architecture perspective, this endpoint design creates the minimum useful interface for local model serving.

It is small enough to reason about, but complete enough to support real model-serving behavior.

The Request Contract

A typical generation request includes:

- prompt
- model name
- temperature
- max token limit

These fields represent some of the core controls used in LLM applications.

The prompt is the input text.

The model name tells the runtime which model to use.

The temperature controls randomness.

The max token limit controls response length.

The non-streaming response includes important metadata:

- model name
- runtime
- response text
- latency
- tokens per second
- token usage when available

That response structure matters because model-serving APIs should not only return generated text. They should also return operational context.

A platform needs to know what model was used, what runtime handled the request, how long generation took, and how much throughput was achieved.

That information becomes important later for debugging, benchmarking, evaluation, cost analysis, and production monitoring.

A response without metadata may be enough for a demo.

A response with metadata is more useful for infrastructure.

Runtime Abstraction: The Most Important Design Choice

The runtime abstraction is one of the most important ideas in the lab.

The project can run against Ollama for real local inference, but it can also use a built-in mock runtime. That means the API and benchmark path can be tested even before a real local model runtime is installed.

This is a strong engineering decision.

A mock runtime makes the service easier to test. It allows the API contract, health endpoint, generation endpoint, streaming behavior, and benchmark script to be exercised without depending on model availability.

Ollama provides the real local inference path. It gives the developer a fast way to run local models and test the full model-serving flow.

Together, the mock runtime and Ollama integration create a clean separation between the wrapper and the backend.

The API does not need to be rewritten every time the runtime changes.

The runtime client becomes the boundary between platform code and inference engine.

That boundary is what allows the architecture to grow.

Today the backend may be Ollama.

Later it could be llama.cpp.

Later still it could be TGI, vLLM, or a managed inference endpoint.

The wrapper pattern remains stable.

That is platform thinking.

Why Streaming Matters

Streaming is one of the most important user experience patterns in LLM applications.

Without streaming, the user waits until the entire response is complete.

With streaming, the user sees the answer begin as soon as the model starts producing output.

Even if the total generation time is the same, streaming makes the system feel faster.

This matters for chat interfaces, coding assistants, local agents, internal copilots, and developer tools.

The Local LLM Serving Lab supports streaming through `POST /generate/stream`.

The flow is straightforward:

1. The client sends a prompt to the FastAPI wrapper.
2. The wrapper forwards a streaming request to the runtime.
3. The local model tokenizes the prompt and starts inference.
4. Each generated token or chunk is sent back through the API.
5. The client receives incremental output instead of waiting for the full response.

From a user experience perspective, streaming improves responsiveness.

From an infrastructure perspective, streaming changes how the service behaves.

The API must support a long-lived response.

The runtime must expose incremental output.

The client must consume partial chunks.

The platform must eventually understand the difference between first-token latency and full-response latency.

That last point is especially important.

In LLM systems, latency has multiple meanings.

A user cares about when the response starts.

A system owner cares about when the response finishes.

A platform engineer cares about throughput, token generation rate, runtime overhead, and resource utilization.

Streaming forces the architect to think about all of those concerns.

Non-Streaming vs. Streaming Generation

The lab supports both non-streaming and streaming generation because both patterns are useful.

Non-streaming generation is simpler. The client sends a prompt and waits for a complete response. This is useful for batch-style operations, simple API calls, tests, and workflows where the full response is needed before the next step can begin.

Streaming generation is more interactive. The client receives chunks as they are produced. This is useful for chat, code generation, agent interfaces, and any experience where perceived latency matters.

A platform should understand both.

Non-streaming is easier to benchmark and reason about.

Streaming is closer to the interaction pattern users expect from modern LLM tools.

The Local LLM Serving Lab makes both paths explicit, which is the right design choice for learning model-serving infrastructure.

Benchmarking: Turning Local Inference into Measurable Infrastructure

One of the most important parts of this lab is the benchmark tooling.

Without benchmarks, model-serving decisions are mostly subjective.

A model may feel fast with one prompt and slow with another.

A runtime may appear responsive with short outputs but struggle with longer completions.

A quantized model may run quickly but produce lower-quality answers.

A larger model may give better responses but require more memory and generate fewer tokens per second.

The benchmark script gives the lab a repeatable way to measure performance.

It sends repeated requests to the local API and records results to JSON and CSV files under `benchmark-runs/`.

The key metrics include:

Metric	What It Tells You
Average latency	Typical end-to-end request time
P50 latency	Median request latency
Max latency	Slowest request in the benchmark set
Tokens per second	Completion throughput reported by the runtime
Completion tokens	Number of generated tokens
Response characters	Rough sanity check for output size

These metrics help move the conversation from “this model feels fast” to “this model produced this throughput under these conditions.”

That shift is critical for AI infrastructure.

Performance needs to be measured, not guessed.

Latency Has Multiple Meanings

The lab highlights an important serving lesson: latency is not one number.

End-to-end latency measures the full request time.

First-token latency measures how quickly the model begins responding.

Prompt evaluation time measures how long the runtime spends processing the input prompt.

Generation latency measures how long it takes to produce output tokens.

Tokens per second measures throughput.

Each metric answers a different question.

If a chat interface feels slow, first-token latency may be the issue.

If long answers take too much time, tokens per second may be the issue.

If large prompts slow everything down, prompt evaluation time may be the issue.

If users experience inconsistent performance, max latency or tail latency may be the issue.

An AI Data Platform Architect needs to understand these distinctions because model-serving performance cannot be managed with a single average.

The lab introduces this mindset by measuring latency and throughput directly.

Runtime Tradeoffs

The project documents four runtime paths:

Runtime	Best Use
Ollama	Fast local experimentation and simple model management
llama.cpp	Understanding quantization, GGUF models, and CPU/Apple Silicon inference
Hugging Face TGI	Production-style GPU serving and batching
vLLM	High-throughput serving concepts like PagedAttention

This comparison is important because there is no universal best runtime.

Ollama is the right default for this lab because it provides the shortest path to working local inference. It is developer-friendly and well-suited for local experimentation.

llama.cpp is valuable because it exposes lower-level concepts around quantization, GGUF models, CPU inference, and Apple Silicon inference. It is useful for understanding how local inference works closer to the metal.

TGI introduces more production-style serving concepts, particularly around GPU-backed serving and batching.

vLLM introduces high-throughput serving concepts and more advanced serving efficiency.

The point is not to declare a winner.

The point is to understand tradeoffs.

A local developer assistant, a batch evaluation runner, a RAG application, and a production inference API may all have different serving requirements.

The platform architecture should leave room for those requirements to change.

Quantization and Model Size

Quantization is another core topic in local inference.

Smaller models and lower-precision formats can reduce memory usage and improve speed. That makes local inference more practical on laptops and smaller machines.

But quantization is not free.

It can affect output quality.

The right tradeoff depends on the workload.

For simple summarization, classification, or local developer experiments, a smaller quantized model may be enough.

For more complex reasoning, coding, or domain-specific analysis, a larger model may perform better but require more memory and deliver lower throughput.

The benchmark tooling gives the developer a way to observe these tradeoffs instead of guessing.

This is exactly the kind of thinking required in AI platform design: performance, quality, and resource use must be considered together.

Context Windows and Prompt Shape

Context window size also affects model-serving behavior.

A short prompt and a long prompt do not place the same demand on the runtime.

A small context window may be fine for simple chat.

A larger context window may be required for document analysis, code understanding, RAG, or multi-turn workflows.

But larger context windows can increase processing time and memory pressure.

Prompt length, output length, model size, runtime, and hardware all affect benchmark results.

That is why benchmarking needs context.

A latency number by itself is not very useful unless the platform knows what model, prompt, output length, runtime, and hardware produced that number.

The Local LLM Serving Lab correctly frames benchmarking as a contextual exercise, not a universal leaderboard.

Local Model Serving Is Still Service Design

One of the strongest lessons from this lab is that local model serving is still service design.

Even on a laptop, a model-serving system benefits from:

- API contracts
- health checks
- runtime configuration
- request validation
- error handling
- streaming behavior
- benchmark tooling
- response metadata
- generated documentation

This is the difference between experimenting with a model and building an inference service.

A local model can be useful on its own.

A local model behind a well-designed API becomes a platform component.

That component can later connect to RAG pipelines, agent workflows, observability traces, load tests, and deployment automation.

The lab is small, but the shape is correct.

Why the Mock Runtime Matters

The mock runtime is easy to overlook, but it is an important part of the lab.

A mock runtime allows the serving wrapper to be tested without requiring a real local model to be installed or loaded.

This makes development faster.

It also makes the API easier to validate.

A developer can test endpoints, response shapes, benchmark scripts, and streaming paths before debugging runtime-specific issues.

That separation is a core software engineering practice.

It prevents the model runtime from becoming a hard dependency for every test.

It also gives the project a cleaner architecture: the API layer can be developed independently from the inference backend.

For platform work, that independence matters.

How This Becomes a Foundation for RAG

The Local LLM Serving Lab is not a full RAG system, but it creates a key foundation for one.

A RAG system needs a generation layer.

That generation layer should be callable through an API.

It should support configurable models.

It should handle full responses and streaming responses.

It should expose health checks.

It should produce metrics that help evaluate performance.

The lab provides those pieces.

Later, retrieval can be added in front of generation.

A RAG request might retrieve relevant chunks from a vector store, assemble a context-rich prompt, call the local model-serving API, stream the response back to the user, and record latency and throughput.

The model-serving wrapper becomes the generation service inside the larger RAG architecture.

That is the right progression: first build the serving layer, then connect retrieval, memory, evaluation, and observability.

How This Becomes a Foundation for Observability

The benchmark script is a first step toward observability.

It records latency and throughput in a repeatable format.

In a future version, those measurements could evolve into runtime metrics, dashboards, traces, and alerts.

Useful serving observability could include:

- request count
- error count
- average latency
- p50 latency
- p95 latency
- p99 latency
- first-token latency
- tokens per second
- prompt tokens
- completion tokens
- model name
- runtime name
- hardware profile
- streaming vs. non-streaming usage

These metrics would help answer practical operating questions.

Which model is fastest?

Which runtime is most stable?

Which prompt types are slowest?

Which workloads produce the most tokens?

Where does latency come from?

Does streaming improve perceived responsiveness?

Are longer context windows causing degradation?

The Local LLM Serving Lab does not need to implement the full observability stack immediately. But by starting with benchmark outputs, it builds the measurement habit early.

That habit is essential for AI infrastructure.

How This Becomes a Foundation for Kubernetes

The lab is intentionally local-first, which is the right place to start.

Before introducing distributed infrastructure, it is better to understand the core serving behavior locally.

Once the wrapper, runtime abstraction, streaming path, and benchmarks are working, the architecture can evolve toward containerized and orchestrated deployment.

A future Kubernetes version could separate:

- API wrapper pods
- model runtime pods
- benchmark jobs
- observability sidecars
- runtime-specific deployments
- GPU-backed nodes
- autoscaling policies

But Kubernetes should not be the starting point.

The starting point is understanding the service contract.

The Local LLM Serving Lab does that.

It creates a model-serving layer that can later be deployed, scaled, observed, and hardened.

The AI Data Platform Architect View

From an AI Data Platform Architect's perspective, this lab represents the model-serving layer of a broader AI platform.

That layer has several responsibilities.

First, it abstracts inference behind an API. Applications should not need to know the details of the local runtime.

Second, it supports multiple interaction patterns. Some workloads need full responses. Others need streaming.

Third, it measures performance. Latency and throughput are part of the system contract.

Fourth, it supports runtime comparison. Ollama, llama.cpp, TGI, and vLLM each represent different serving tradeoffs.

Fifth, it prepares for future platform layers. RAG, observability, performance testing, and Kubernetes deployment all require a serving foundation.

This is the architectural value of the lab.

It turns local inference into a service.

It turns a model into an API-backed platform component.

It turns subjective performance into measurable output.

It turns a laptop experiment into the first version of a model-serving architecture.

Final Takeaway

The `03-local-llm-serving-lab` is a practical step between running a local model and designing a production model-serving platform.

It shows how to wrap local inference behind FastAPI, connect to Ollama, use a mock runtime for smoke tests, expose non-streaming and streaming generation endpoints, compare serving runtimes, and benchmark latency and tokens per second.

The biggest lesson is that model serving is not just about generating text.

It is about API design, streaming user experience, runtime abstraction, latency, throughput, quantization, context windows, and repeatable measurement.

For an AI Data Platform Architect, this project builds the serving layer that future systems can depend on.

Before adding RAG, agents, observability, load testing, or Kubernetes deployment, the platform needs a clean way to call a model, stream output, measure performance, and reason about runtime tradeoffs.

That is what this lab provides.